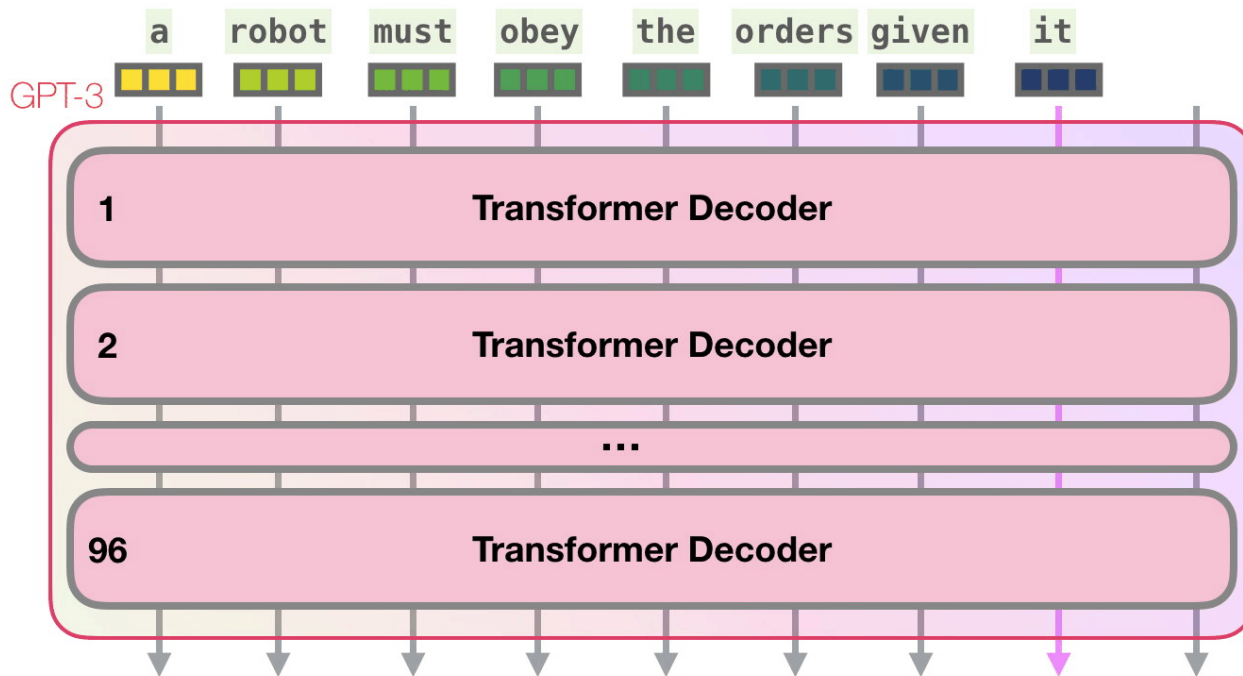




Transformers: Advanced

Современные архитектуры нейронных сетей

Recap of previous lesson: LLM

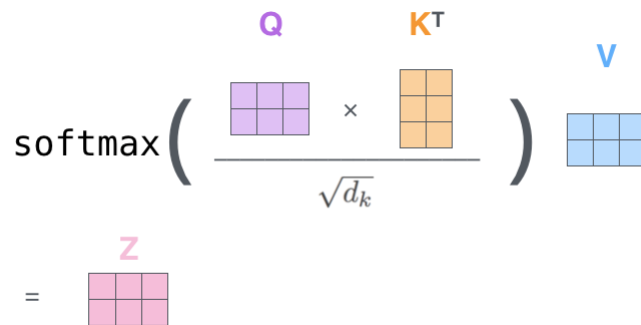


Recap of previous lesson: Self-attention

$$X \times W^Q = Q$$


$$X \times W^K = K$$


$$X \times W^V = V$$

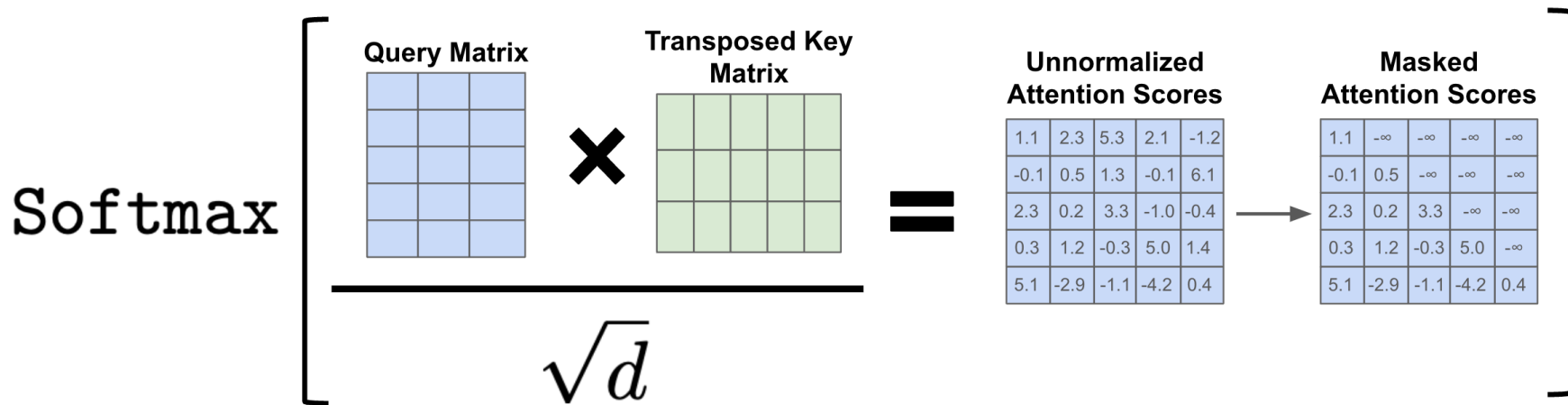

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$


Q (Query) — запрос токена к другим токенам.

K (Key) — ключевые признаки токена для сопоставления.

V (Value) — содержимое токена, которое используется для формирования итогового представления.

Recap of previous lesson: Masked Self-Attention



План лекции

- 1) Scaling Laws
- 2) Ускорение инференса
 - а) KV-cache
 - б) Speculative Decoding
- 3) Архитектурные трюки
 - а) Grouped Query Attention (GQA) and Multi Query Attention (MQA)
 - б) Sliding Window attention
 - в) Sparse Attention
 - г) Mixture of Experts (MoE)
 - д) Multi-Head Latent Attention (MLA)
- 4) Дообучение моделей
 - а) LoRa



Scaling Laws (2021)

- **Размер модели (N):** от 768 до 1,5 миллиардов настраиваемых параметров.
- **Размер набора данных (D):** от 22 миллионов до 23 миллиардов токенов.
- **Потеря кросс-энтропии на тесте (L)**
- **Объем вычислений (C)**— использованных для обучения модели. Измеряется в PF-days



Что такое PF-days?

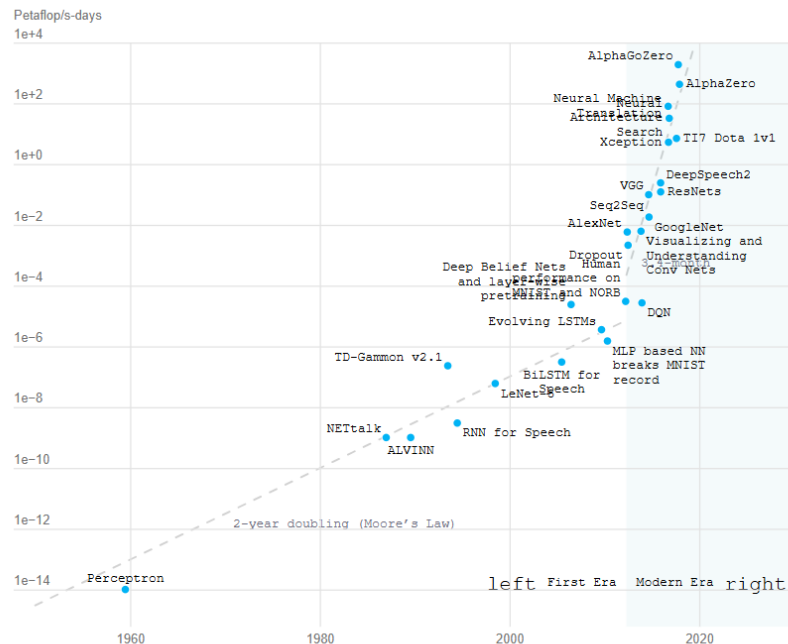
FLOP = *floating point operation* (операция с плавающей точкой).

FLOP/s = скорость, сколько таких операций в секунду.

petaFLOP/s (1 PF-sec) = 10^{15} операций в секунду.

PF-day = 10^{15} FLOP/s $\times$ (24 $\times$ 3600)секунд $\approx 10^{20}$ FLOP

PF-day and models

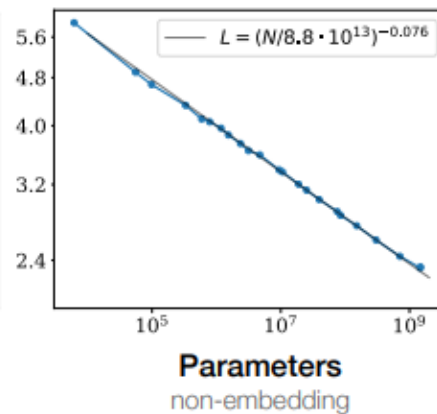
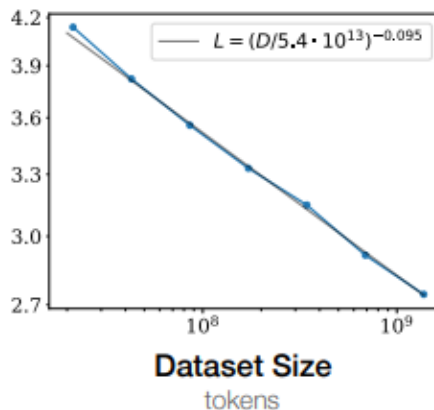
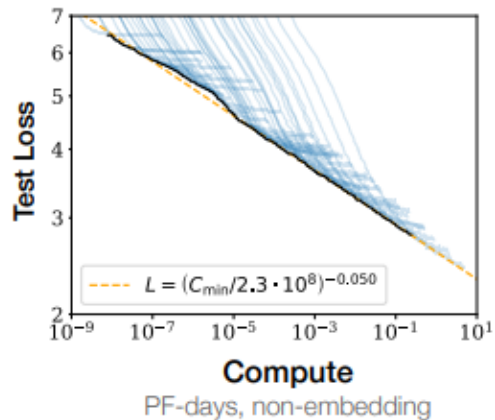


FP32 (TFLOPS)

Per graphics card

	GIGABYTE AORUS GeForce RTX 5090 STEALTH ICE 32 GB GDDR7	115.60
	GIGABYTE AORUS GeForce RTX 5090 MASTER 32 GB GDDR7	115.60
	GIGABYTE AORUS GeForce RTX 5090 MASTER ICE 32 GB GDDR7	115.60
	GIGABYTE AORUS GeForce RTX 5090 XTREME WATERFORCE 32 GB GDDR7	115.60
	GIGABYTE AORUS GeForce RTX 5090 XTREME WATERFORCE WB 32 GB GDDR7	115.60
	ASUS ROG Matrix GeForce RTX 5090 30th Anniversary Edition 32 GB GDDR7	113.64

Scaling Laws (2021)



Фиксируем

Размер модели

Размер модели

Количество мощностей

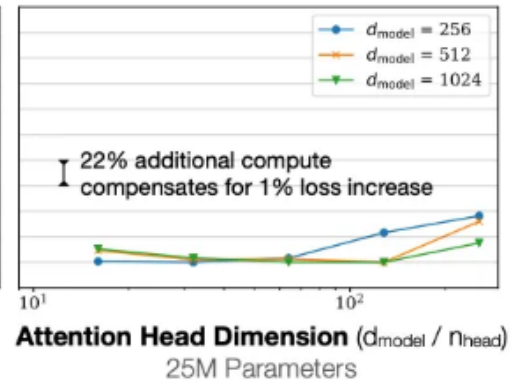
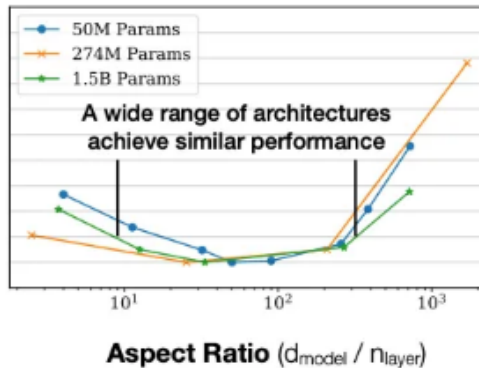
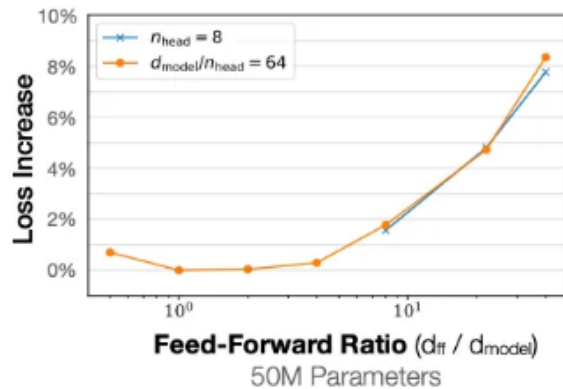
Проверяем достаточно

Данных

Количество мощностей

Данных

Scaling Laws (2021)



Что хотим сделать?

- У нас есть выделенное C (PF-дней)
 - Например у нас есть кластер из R карточек на K дней
- Как обучить наилучшую модель за это время? (что такое наилучшая модель?)
- Будем искать модель с наименьшим лоссом L (Как мы выяснили, L зависит от D и N)
- Тогда решаем задачу такую:

$$N_{opt}(C), D_{opt}(C) = \underset{N, D \text{ s.t. } FLOPs(N, D) = C}{\operatorname{argmin}} L(N, D)$$

Scaling Laws (2021)

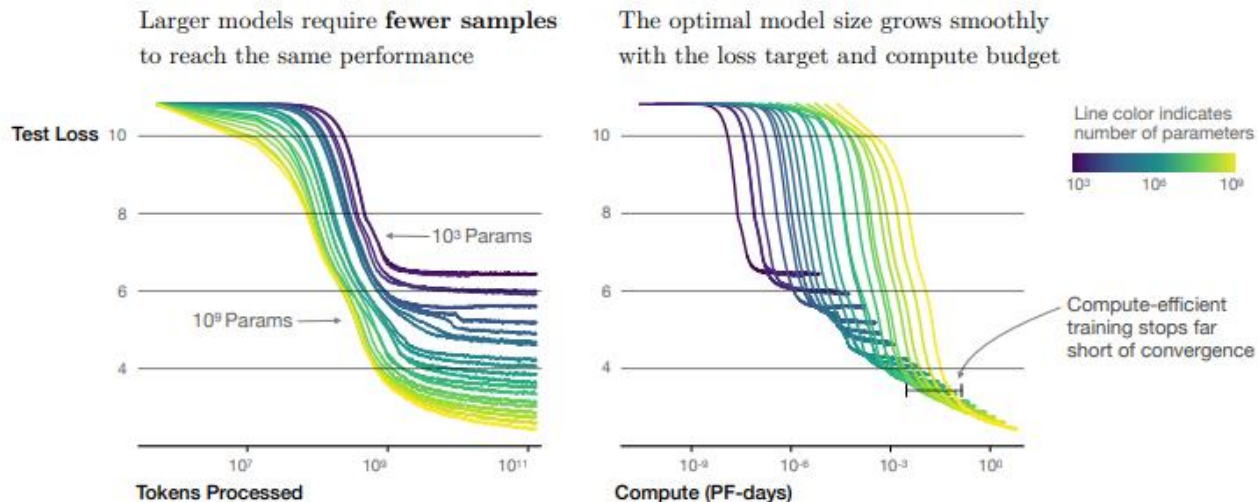


Figure 2 We show a series of language model training runs, with models ranging in size from 10^3 to 10^9 parameters (excluding embeddings).

Scaling Laws (2021)

Самый главный вывод статьи:



Увеличиваем Compute	Что делать с параметрами	Почему
$C \uparrow \times 10$	$N \uparrow \times 5.4$	Модель должна стать крупнее, иначе compute будет тратиться неэффективно
	$D \uparrow \times 1.9$	Нужно немного больше данных, чтобы не уйти в overfit
	$L \downarrow \times 0.89$	Потери падают на $\approx 11\%$

Scaling Laws: последствия и предпосылки ИТМО

Модель	Параметры	Данные	Как обучали
GPT 2 (2019)	117 млн	1) 7000 книг 2) ?	1) Next token prediction 2) SFT
GPT-3 (2020)	117 млрд	1) 70000 книг	1) Next token prediction
InstructGPT (2022)	117 млрд Основано на GPT-3	2) ? 3) ?	2) SFT 3) RLHF
GPT-4 (2023)	По слухам 1.8 трлн	?	1) NTP 2) SFT + RLHF 3) Multimodal 4) Safety, alignment
GPT-5 (2025)	?	?	?



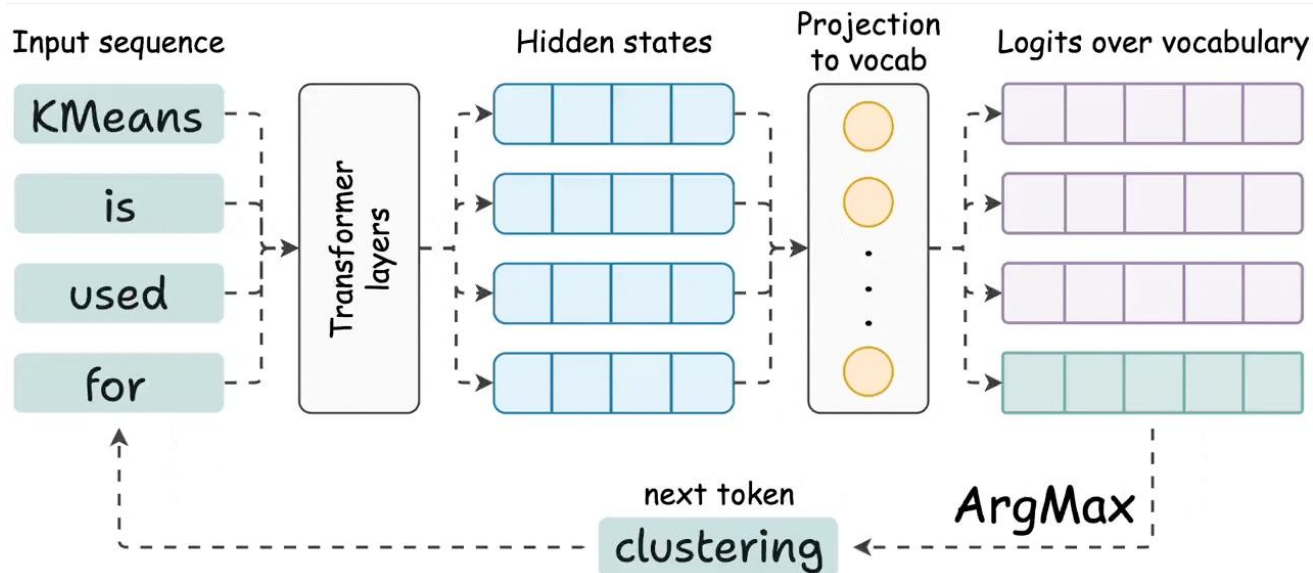
Увеличение скорости работы LLM

- 1) KV-cache
- 2) Speculative Decoding

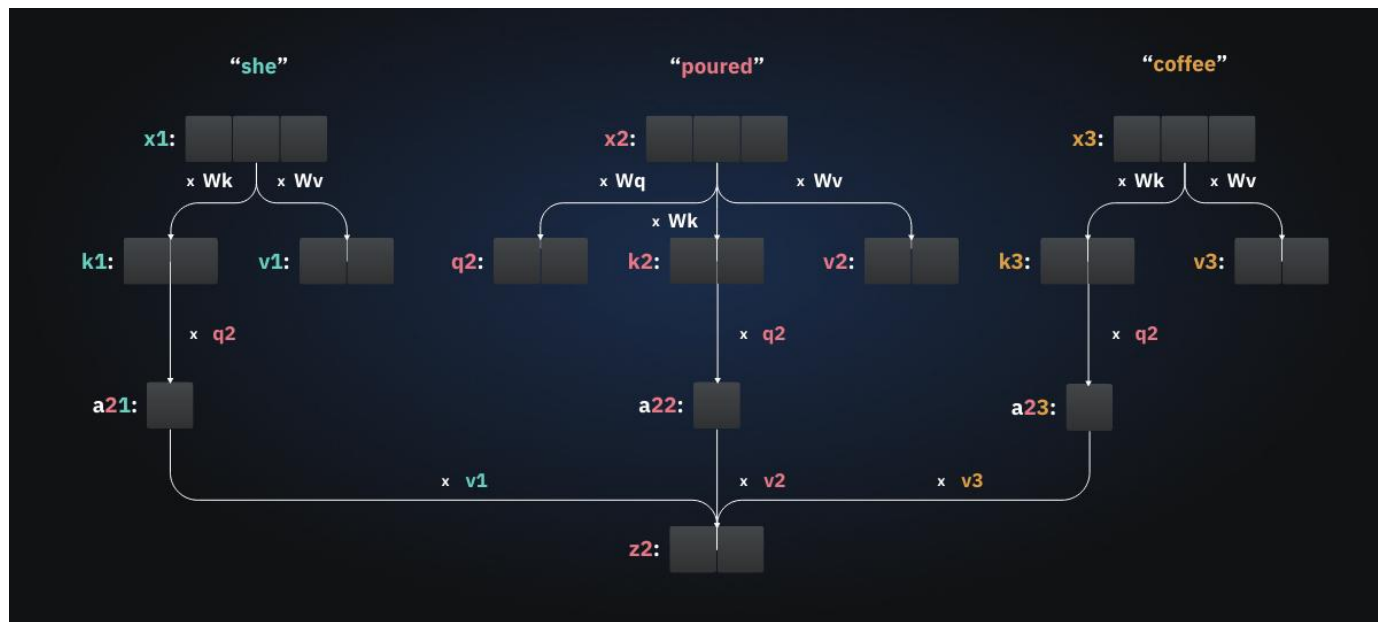


Работают с любой обученной LLM!

Как работают LLM



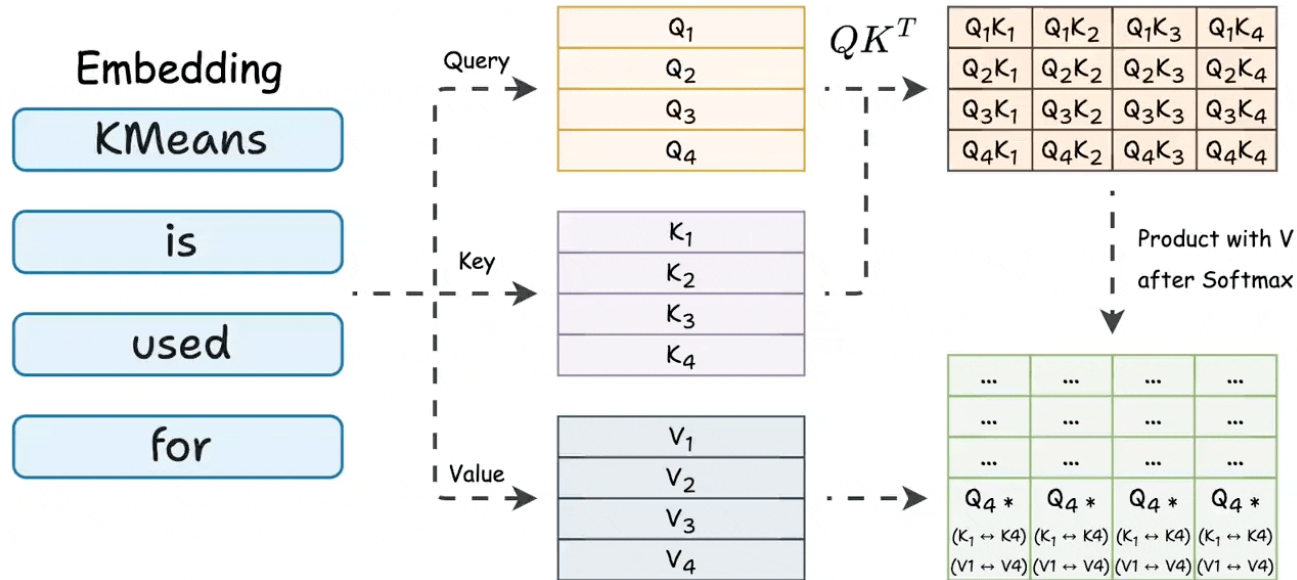
Self-Attention



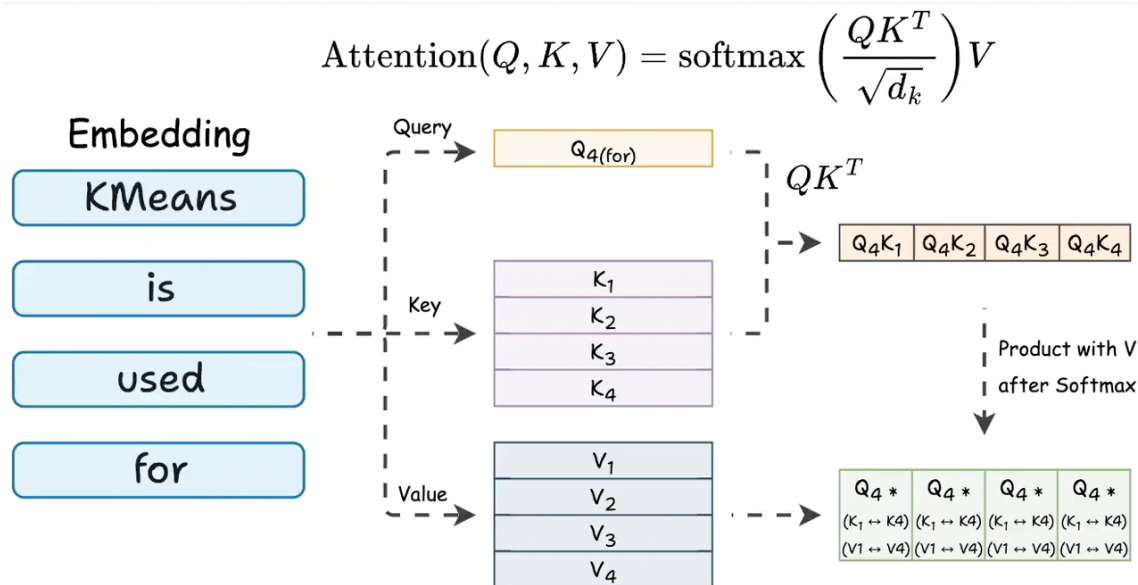
Self-Attention



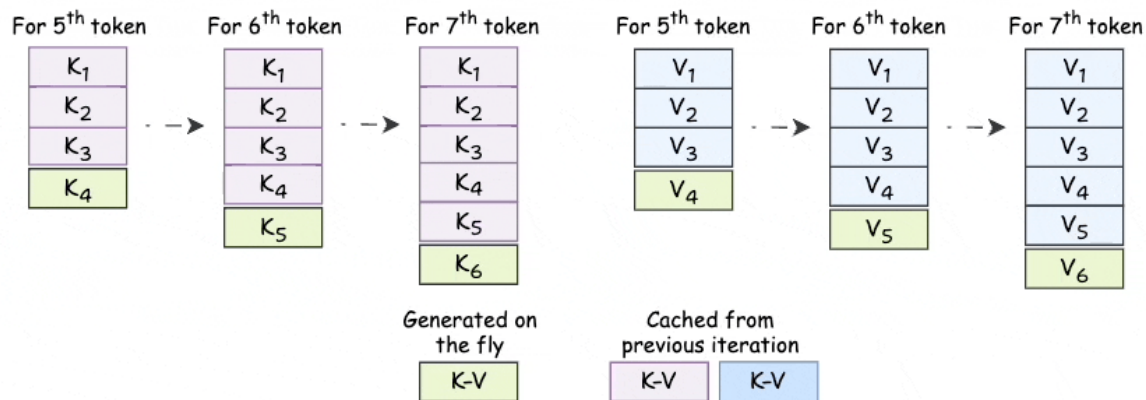
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



Self-Attention last token

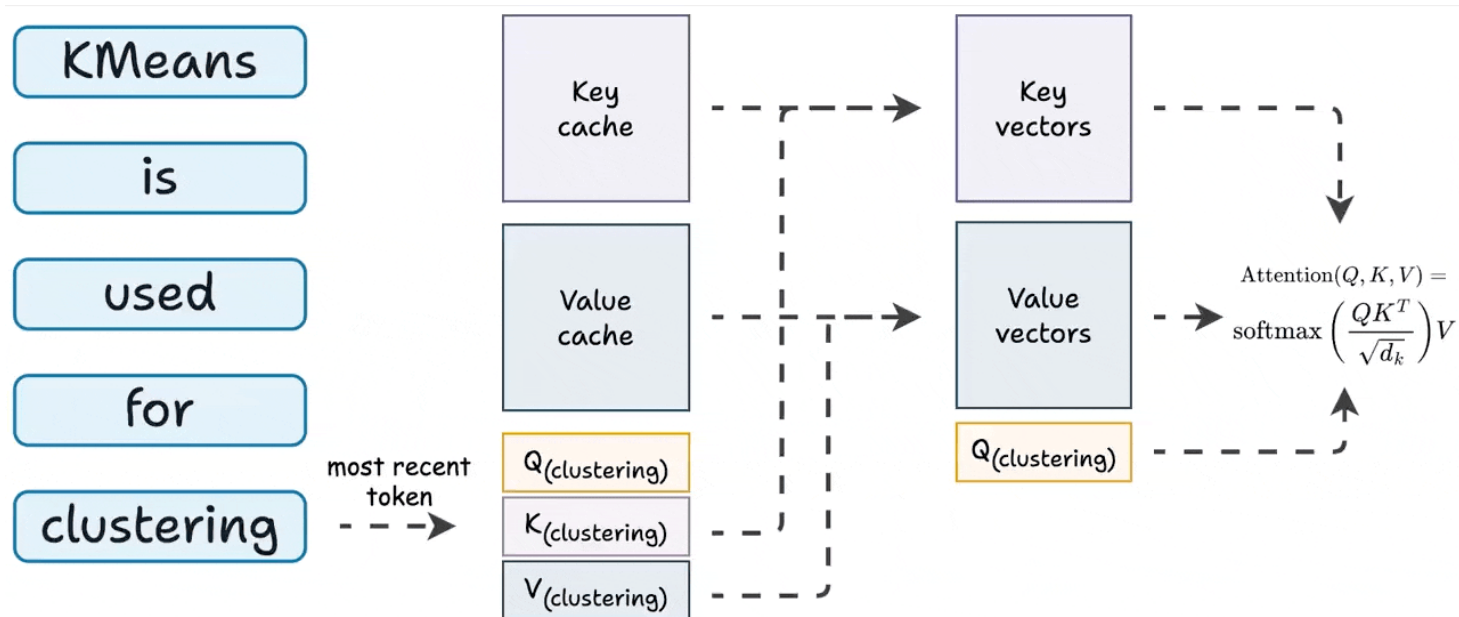


Self-Attention last token



The **key** vectors and **value** vectors used during previous tokens do not change. Cache them to avoid recomputing them.

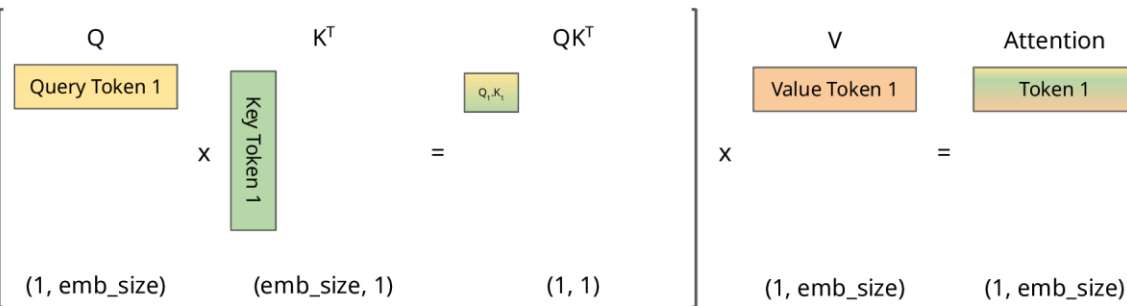
KV-cache



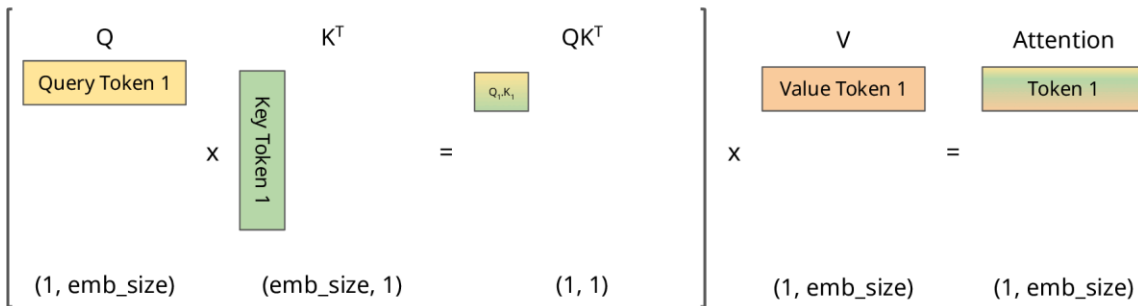
KV-cache

Step 1

Without
cache



With
cache



□ Values that will be masked ■ Values that will be taken from cache



KV-cache



with KV-cache	Without KV-cache
Хранится K,V на каждый токен для каждого слоя	Хранится только текущие активации (всё пересчитывается)
БОЛЬШЕ затрат по памяти GPU	МЕНЬШЕ затрат по памяти GPU
БЫСТРЕЕ генерация токенов	МЕДЛЕННЕ генерация токенов
Так как на каждом слое вычисляются только вектора	Так как на каждом слое вычисляются матрицы

Стратегии KV-cache

Тип	Описание	Плюсы/Минусы	Память
DynamicCache (по умолчанию)	Кэш растёт динамически по мере генерации. Это стандарт для всех autoregressive моделей.	Простота, гибкость не оптимален для JIT	● Средняя
StaticCache	Кэш фиксированного размера — заранее аллоцируется под максимум токенов	Можно компилировать (torch.compile), быстро при одинаковой длине батчей Расточителен — лишние токены маскируются, не участвуют в attention	● Высокая
OffloadedCache (Dynamic / Static)	KV кеш слоёв выгружается с GPU на CPU, остаётся только текущий слой	Позволяет обрабатывать длинные контексты на слабых GPU Медленнее — данные гоняются между CPU и GPU	● Средняя–низкая
EncoderDecoderCache	Отдельно кэширует encoder и decoder (для seq2seq моделей, типа T5, BART)	Удобно, гибко, нативно для encoder-decoder архитектур Только для таких моделей, не для чистых decoder-only	● Средняя

Проблема: последовательность и задержки в LLM

Автогенерация текста в LLM — **строго последовательная**: каждый токен требует отдельного прохода через модель.



Из-за этого:

- GPU простаивает, т.к. вычислительные блоки не могут работать параллельно;
- каждый шаг = отдельная синхронизация и загрузка весов;
- высокая латентность (задержка ответа).

Вывод: даже при мощных GPU производительность ограничивается архитектурой автогенерации.

Что такое speculative decoding

Спекулятивное декодирование — это оптимизация, позволяющая **генерировать несколько токенов за один проход**.

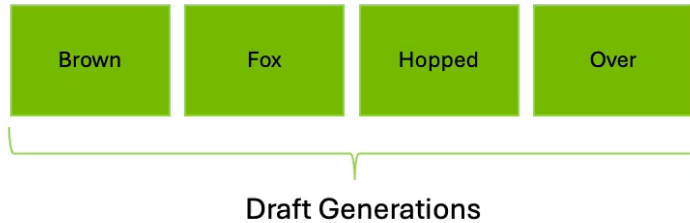


Принцип:

- 1) Есть **большая (target)** модель и **лёгкая (draft)** модель.
- 2) Draft быстро предлагает несколько следующих токенов (спекуляции).
- 3) Target проверяет их за один форвард-проход.
- 4) Принимаются только те токены, которые target тоже предсказала бы сама.
- 5) Остальные — отбрасываются, и генерация продолжается с последнего принятого.

✅ Итог: модель генерирует n токенов за 1 forward pass, сохраняя точность target-модели.

Speculative decoding



Механика draft–target подхода

Draft generation

- Маленькая модель (или head) генерирует 3–12 токенов.
- Обучается на тех же данных, где таргет служит ground truth.

Parallel verification

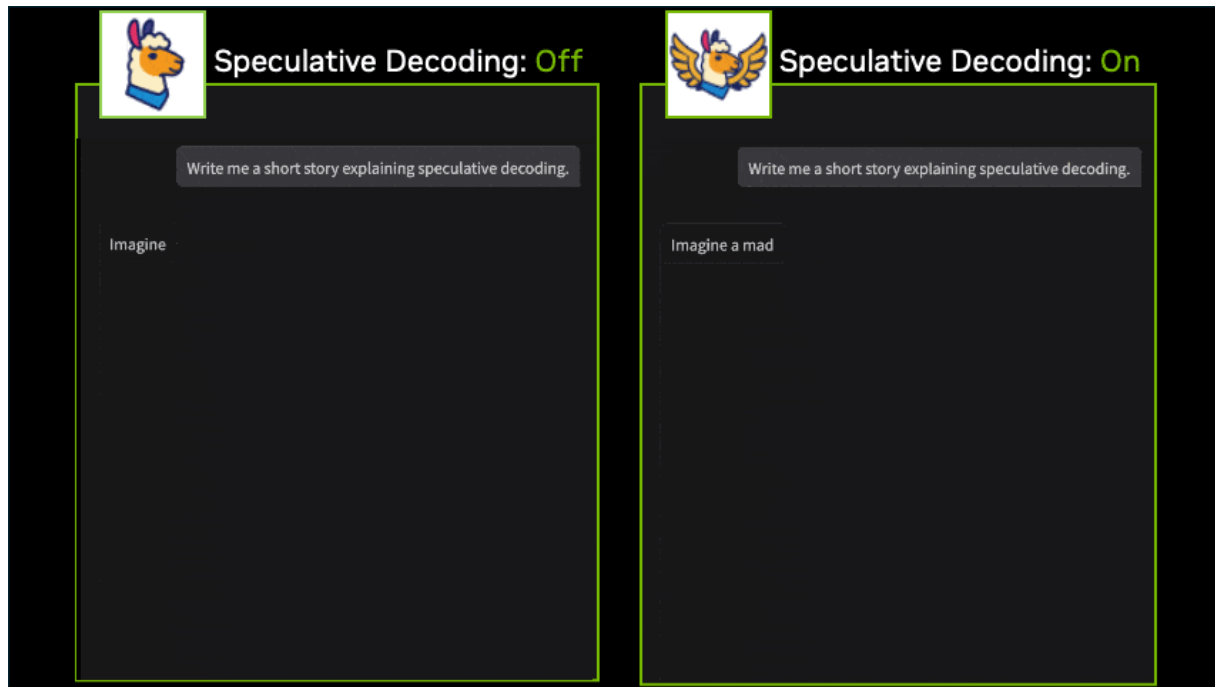
- Target обрабатывает все draft-токены параллельно.
- Благодаря KV Cache повторно вычислять старые токены не нужно — только новые.
- Экономия на последовательных шагах = ускорение в 2–4 раза.

Rejection sampling

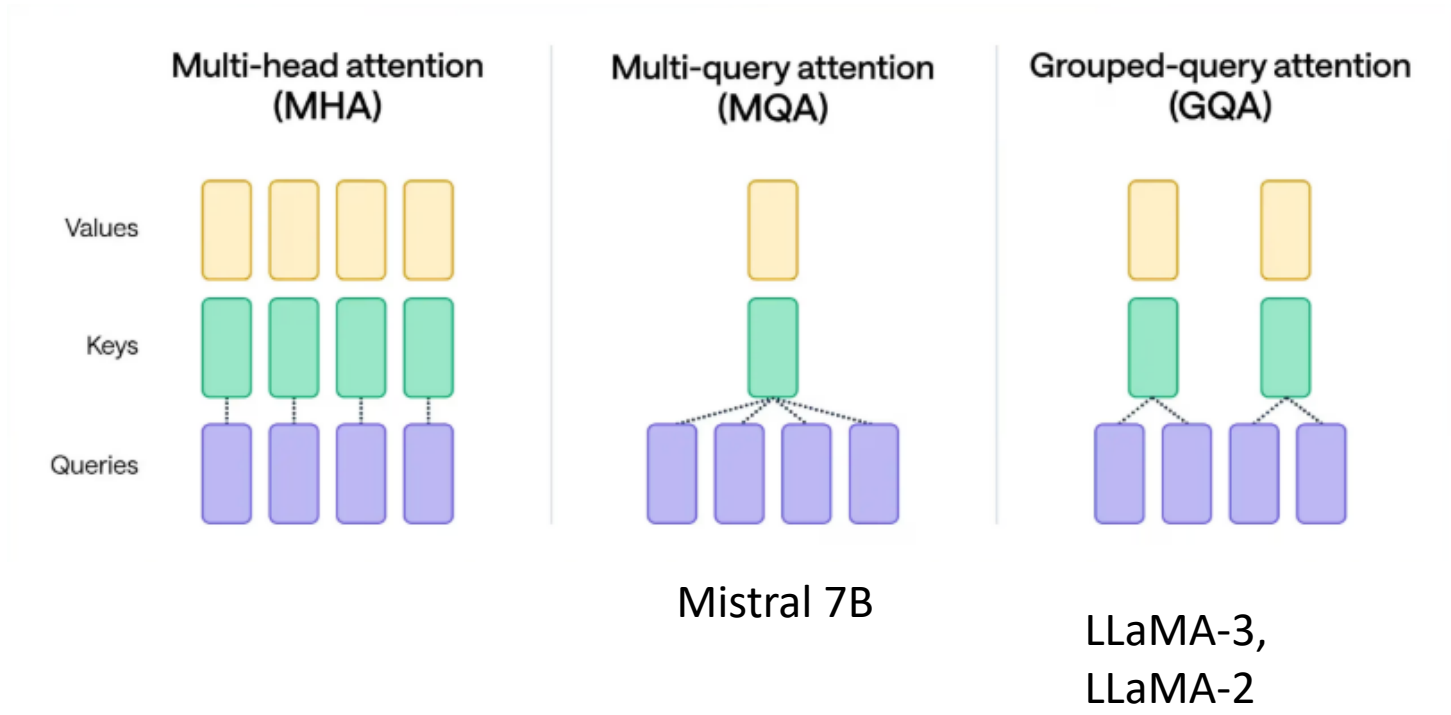
- Для каждого токена сравниваются вероятности $P(\text{Target})$ и $P(\text{Draft})$.
- Если $P(\text{Target}) \geq P(\text{Draft}) \rightarrow$ токен принимается.
- При первом несовпадении отклоняются все последующие draft-токены.



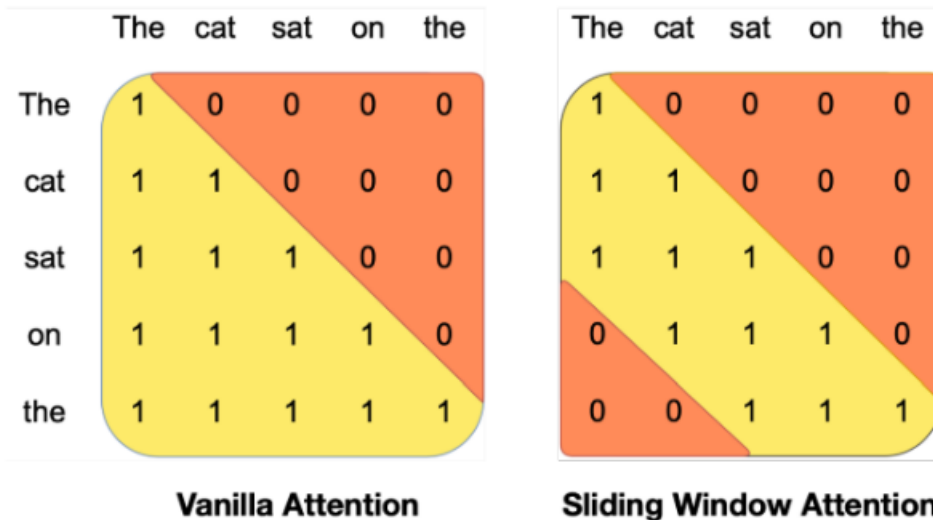
Speculative decoding



GQA and MQA

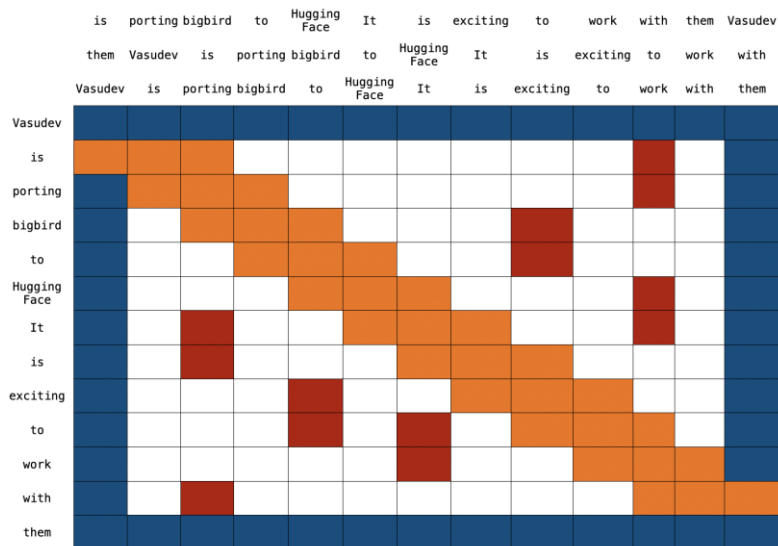


Sliding window attention



Mistral 7B, Claude 3 (Anthropic)

BigBird's Block Sparse Attention

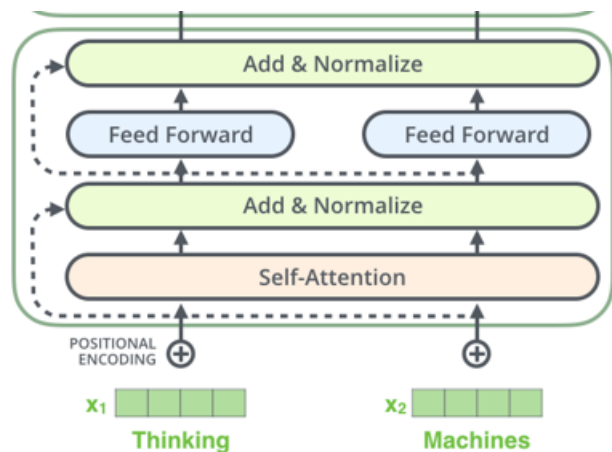


Global

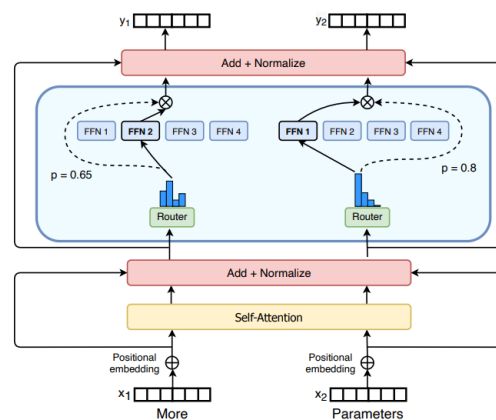
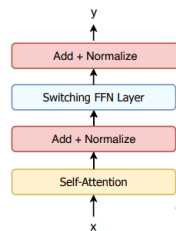
Sliding

Random

Mixture of Experts (MoE)



Transformer



MoE
Deepseek-R1

Multi-Head Latent Attention (MLA)

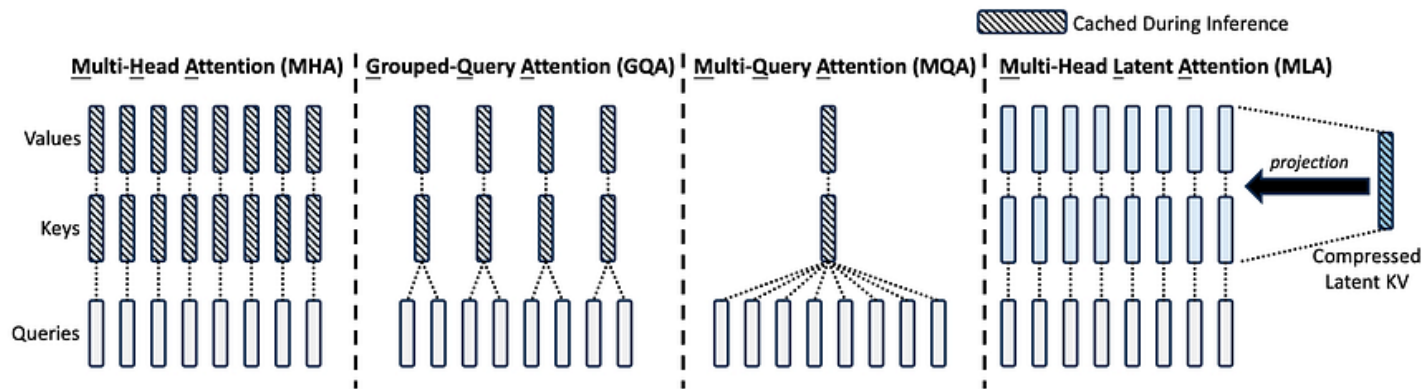


Figure 3 | Simplified illustration of Multi-Head Attention (MHA), Grouped-Query Attention (GQA), Multi-Query Attention (MQA), and Multi-head Latent Attention (MLA). Through jointly compressing the keys and values into a latent vector, MLA significantly reduces the KV cache during inference.

1) Модель слишком большая, тонкая настройка — дорогая

Большие языковые модели (LLM) содержат миллиарды параметров. fine-tuning требует менять все параметры, что очень ресурсоёмко и медленно.

2) Производительные и финансовые ограничения

При классическом подходе ты сталкиваешься с высокими требованиями к GPU-памяти, времени обучения и инфраструктуре.

Что такое LoRA (Low-Rank Adaptation)

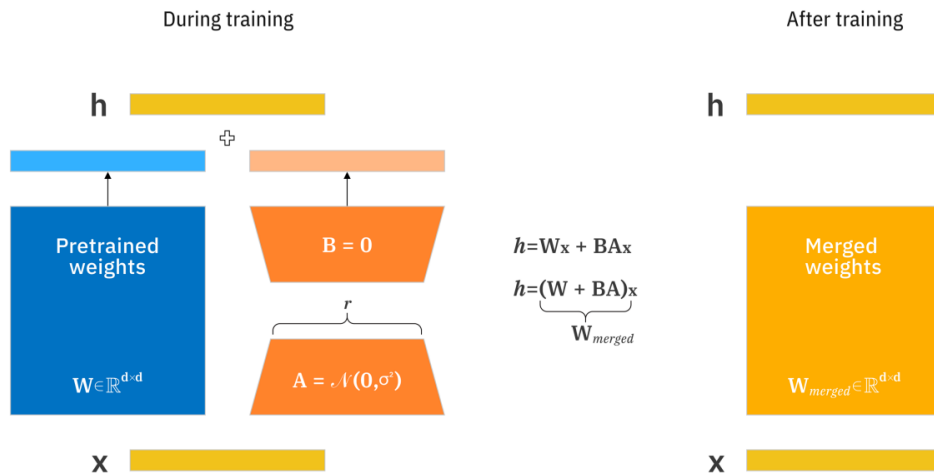


LoRA — метод адаптации (тонкой настройки) больших моделей путём **добавления лёгких адаптеров**, а не изменения всей модели.

Основная модель **фиксируется** (“замораживается”), и модификации вносятся через новые **низкоранговые матрицы**, которые обучаются отдельно.

Суть — выразить изменения, нужные для адаптации, через небольшие (минимальные) матрицы, вместо полной корректировки огромных весов.

Что такое LoRA (Low-Rank Adaptation)



Как работает LoRA — механизм

Введение низкоранговых матриц A и B



Для каждого слоя (или в выбранных модулях, например, блоках внимания) вводят две матрицы меньшего ранга. Эти матрицы создают «дельту» (изменения), которую добавляют к исходным весам.

Заморозка исходных весов

Все параметры базовой модели остаются нетронутыми.

Обучение этих адаптеров

Обучаем только A и B (их параметры) через градиентный спуск, остальные веса не изменяются.

Слияние или применение изменений

При использовании модели (инференсе) можно «слить» (merge) адаптерные матрицы с основными весами, чтобы не было дополнительной задержки при выводе.

Параметры настройки LoRA

- **r**: ранг адаптерных матриц (насколько «широко» они модифицируют модель)
- **target_modules**: модули, к которым применять адаптеры (не обязательно ко всем слоям)
- **lora_alpha**: масштабный коэффициент, регулирующий силу адаптации

Преимущества

ІТМО

Снижение числа обучаемых параметров

Вместо десятков миллиардов параметров — лишь миллион-два для адаптеров.



Меньше требований к GPU и памяти

Поскольку оригинальные веса не участвуют в градиентных вычислениях, отпадает необходимость хранить градиенты и состояния оптимизатора для них.

Гибкость модульности

Один «фризнутый» базовый вес + множество адаптеров для разных задач. Можно просто переключать адаптеры, не храня лишних копий всей модели.

Нет накладных расходов в инференсе (после слияния)

После объединения адаптеров с базовыми весами модель работает с прежней скоростью без ухудшений.

Комбинируемость

Можно совмещать LoRA с другими техниками тонкой настройки для лучшего результата.

Потери при аппроксимации

Поскольку изменение представляется через низкоранговые матрицы, часть информации может быть “сглажена” или утрачена.

Подходящий ранг критичен

Если ранг слишком низкий → модель не сможет выразить нужные изменения; если слишком высокий → теряется смысл “меньшего адаптера”. Нужно балансировать.

Не универсально оптимален

Для некоторых задач может оказаться, что обычная тонкая настройка даёт лучший результат (особенно если задача сильно отличается от домена базовой модели).

Зависимость от избыточности базовой модели

Эффективность LoRA возможна благодаря тому, что большие модели часто переизбыточны (overparameterized).

Последние новости: LoRa



Thinking Machines: 30.09.2025

Главная идея исследования: LoRA может обучаться почти как полный fine-tuning, но при этом быть проще, дешевле и предсказуемее.

Tinker — первый продукт Thinking Machines. 1.10.2025

.Tinker это облачное API для файнтюна LLM направленное на ресёрчеров. Доступно только LoRa дообучение

**Спасибо
за внимание!**

it'sMO *re than a*
UNIVERSITY

Ваши контакты